SIMATS ENGINEERING
ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A

COURSE OUTCOME COVERED
CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 20%

QUESTIONS:5 Total Marks 20 Duration :60 Minutes Annexure A
Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I D.HUSSAINAIAH(192425300)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D.HUSSAINAIAH

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and Recognition Pipeline

A system must process a live video stream to:

- A. Capture frames from a camera
- B. Detect faces in each frame
- C. Recognize faces using a pre-trained dataset
- D. Display bounding boxes and labels in real time

Constraints:

- Must operate continuously
- Should minimize processing delay
- Should handle multiple faces

Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure.

Answer — Pseudocode:

```
BEGIN
  LOAD pretrained face detector (Haar Cascade / DNN model)
  LOAD known face dataset (encodings and labels)
  INITIALIZE video capture from camera

  WHILE camera is active DO
     CAPTURE frame from video stream
     IF frame is empty THEN CONTINUE

     CONVERT frame to grayscale          // faster detection
```

```
    faces = DETECT_FACES(frame)            // supports multiple faces

    FOR EACH face IN faces DO
      ROI = CROP(frame, face.bounding_box)
      ROI = PREPROCESS(ROI)                // resize, normalize
      embedding = EXTRACT_FEATURES(ROI)
      label, confidence = MATCH(embedding, known_dataset)

      IF confidence >= THRESHOLD THEN
        DRAW bounding box on frame
        DISPLAY label and confidence
      ELSE
        DRAW bounding box
        DISPLAY "Unknown"
      END IF
    END FOR

    SHOW frame in real-time window
    IF user presses 'q' THEN BREAK
  END WHILE

  RELEASE camera
  CLOSE all windows
END
```

Design Justification:

The loop reads and displays every frame continuously to satisfy uninterrupted operation. Grayscale conversion and a per-face FOR loop keep per-frame cost low and let the pipeline handle any number of detected faces. A confidence threshold prevents false recognitions.

2. OCR Pipeline for Document Processing

A document processing system must:

- A. Read input images containing printed text
- B. Enhance image quality (noise removal, contrast improvement)
- C. Segment text regions
- D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

Answer — Pseudocode:

```
BEGIN
  LOAD list of input images (batch)

  FOR EACH image IN batch DO
     READ image from disk
     CONVERT image to grayscale

     // --- Enhancement stage ---
     APPLY noise removal (median blur / bilateral filter)
     APPLY contrast enhancement (CLAHE / histogram equalization)
     APPLY adaptive thresholding (binarization)

     // --- Segmentation stage ---
     DETECT text regions using contours / connected components
     SORT regions top-to-bottom, left-to-right (reading order)
```

```
    // --- Text extraction stage ---
    result_text = ""
    FOR EACH region IN sorted_regions DO
        crop = CROP(binarized_image, region)
        text = OCR_ENGINE(crop)          // e.g., Tesseract
        result_text = result_text + text
    END FOR

    SAVE / DISPLAY result_text for this image
  END FOR
END
```

Design Justification:

Noise removal and CLAHE explicitly target the stated constraint of noisy, low-contrast input before any segmentation is attempted, which prevents such defects from propagating into detection errors. Wrapping the whole pipeline in a FOR EACH image loop satisfies the batch-processing requirement, and separating the pipeline into clearly commented Enhancement, Segmentation, and Extraction stages keeps each concern modular and easy to verify independently.

3. Motion Detection and Tracking System

A surveillance system must:

- A. Read video input
- B. Detect motion between consecutive frames
- C. Track moving objects
- D. Highlight detected motion regions

Constraints:

- Should handle dynamic background
- Must operate in near real-time

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

Answer — Pseudocode:

```
BEGIN
  INITIALIZE video capture
  INITIALIZE adaptive background_subtractor (e.g., MOG2)
  tracker_list = empty list

  WHILE video has frames DO
     CAPTURE current_frame
     CONVERT current_frame to grayscale
     APPLY Gaussian blur (reduce noise)

     foreground_mask = background_subtractor.APPLY(current_frame)
     APPLY threshold on foreground_mask
     APPLY morphological dilate/erode (clean mask, merge blobs)

     contours = FIND_CONTOURS(foreground_mask)
```

```
    FOR EACH contour IN contours DO
      IF area(contour) > MIN_AREA THEN
        box = BOUNDING_RECT(contour)
        match = FIND_NEAREST_TRACKER(tracker_list, box)
        IF match found THEN
          UPDATE match.position = box
        ELSE
          CREATE new tracker WITH box
          APPEND new tracker TO tracker_list
        END IF
        DRAW box on current_frame
      END IF
    END FOR

    REMOVE trackers not updated for N frames
    DISPLAY current_frame with highlighted motion regions

    IF user presses 'q' THEN BREAK
  END WHILE

  RELEASE video capture
END
```

Design Justification:

Using an adaptive background subtractor (rather than simple two-frame differencing) allows the model to absorb gradual background variation, directly meeting the dynamic-background constraint. Morphological cleanup and a minimum-area filter remove small noisy blobs cheaply, and nearest-tracker matching maintains object identity frame-to-frame with low overhead, keeping the WHILE loop fast enough for near real-time surveillance.

4. Vehicle Counting System using Video

A traffic monitoring system must:

- A. Read video frames
- B. Detect vehicles entering a region of interest (ROI)
- C. Track movement across frames
- D. Count vehicles and display count

Constraints:

- Avoid double counting
- Handle multiple vehicles simultaneously

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions.

Answer — Pseudocode:

```
BEGIN
  INITIALIZE video capture
  DEFINE region_of_interest (ROI) and virtual counting_line
  INITIALIZE background_subtractor
  tracker_list = empty list
  count = 0

  WHILE video has frames DO
     CAPTURE frame
     roi_frame = EXTRACT(frame, ROI)

     foreground_mask = background_subtractor.APPLY(roi_frame)
     APPLY threshold + morphological cleanup
     contours = FIND_CONTOURS(foreground_mask)

     detections = empty list
```

```
FOR EACH contour IN contours DO
    IF area(contour) > MIN_VEHICLE_AREA THEN
        APPEND BOUNDING_RECT(contour) TO detections
    END IF
END FOR

// --- Association / tracking ---
FOR EACH detection IN detections DO
    match = FIND_TRACKER(tracker_list, detection)   // by IOU/centroid
    IF match found THEN
        UPDATE match.position = detection
        APPEND detection TO match.trajectory
    ELSE
        CREATE new tracker WITH unique_id, detection
        APPEND new tracker TO tracker_list
    END IF
END FOR

// --- Counting logic ---
FOR EACH tracker IN tracker_list DO
    IF tracker.trajectory CROSSES counting_line
      AND tracker.already_counted == FALSE THEN
        count = count + 1
        tracker.already_counted = TRUE
    END IF
END FOR

REMOVE tracker FROM tracker_list IF inactive for N frames
```

```
    DRAW bounding boxes, IDs, counting_line, and count ON frame
    DISPLAY frame
    IF user presses 'q' THEN BREAK
  END WHILE

  RELEASE video capture
END
```

Design Justification:

Each vehicle receives a persistent unique ID from the tracker, and the already_counted flag on that same ID guarantees a single vehicle is only added to the count once, directly satisfying the no-double-counting constraint even as it crosses the line multiple times or momentarily overlaps another vehicle. Processing detections as a list within a single frame (rather than one at a time across separate passes) lets the system handle multiple simultaneous vehicles in the same iteration of the loop.

5. Facial Expression Recognition System

A mobile-based system must:

- A. Capture images from a camera
- B. Detect faces
- C. Extract facial features
- D. Classify expressions (happy, sad, neutral)
- E. Display results

Constraints:

- Limited computational resources
- Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

Answer — Pseudocode:

```
BEGIN
  LOAD lightweight face detector (Haar Cascade)
  LOAD lightweight expression classifier (SVM on HOG / quantized CNN)
  INITIALIZE camera capture

  WHILE camera is active DO
     CAPTURE frame
     RESIZE frame to low resolution        // reduce computation
     CONVERT frame to grayscale

     faces = DETECT_FACES(frame)

     FOR EACH face IN faces DO
        ROI = CROP(frame, face.bounding_box)
        ROI = RESIZE(ROI, fixed_size)      // e.g., 48x48
```

```
        ROI = NORMALIZE(ROI)

        features = EXTRACT_FEATURES(ROI)   // HOG / CNN embedding
        expression, confidence = CLASSIFY(features)

        DRAW bounding box on frame
        DISPLAY expression label and confidence
    END FOR

    SHOW frame
    IF user presses 'q' THEN BREAK
  END WHILE

  RELEASE camera
END
```

Design Justification:

Frame resizing, grayscale conversion, and a lightweight Haar-cascade detector minimize per-frame computation to suit limited mobile resources. Fixing the ROI size and using handcrafted/HOG-style features keeps the classification stage cheap, and structuring the code into clearly separated capture, detection, feature-extraction, and classification steps within a single continuous WHILE loop ensures a modular design that still meets the real-time response requirement.